Université Grenoble Alpes
L3 Informatique

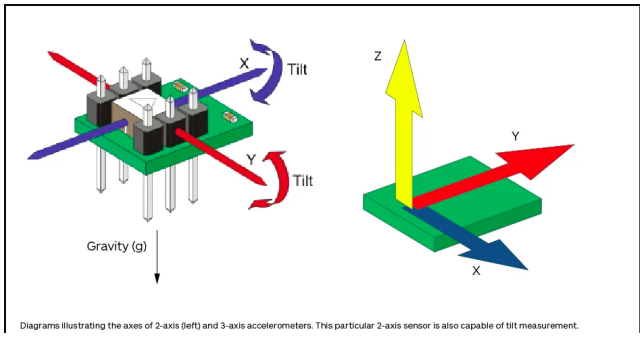
Année 2026-2027

Architectures logicielles et matérielles
Semaine 6 TP

*Nous allons ici expérimenter **l'acquisition de données depuis le périphérique accéléromètre**, accessible via **l'interface SPI**.*
Des réactions sur l'allumage des leds seront faites en fonction des valeurs acquises.

Informations préliminaires. Lire attentivement avant de commencer

• Un **accéléromètre** est un capteur qui, fixé à un mobile ou tout autre objet, sert à mesurer l'accélération de ce dernier dans les 3 axes, x, y et z.



Diagrams illustrating the axes of 2-axis (left) and 3-axis accelerometers. This particular 2-axis sensor is also capable of tilt measurement.

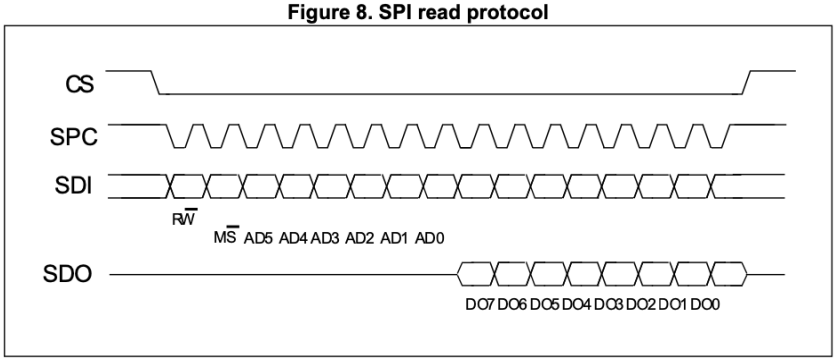
https://blog.knoldus.com/accelerometer-as-puncho-o-meter-the-stm32-discovery-board-sensor-part-1/amp/

Il peut être utile dans divers contextes, par exemple :
- la détection des chocs, comme pour le déclenchement des coussins de sécurité dans les voitures
- la mesure d'accélération, comme pour les stations inertielles des avions
- la mesure d'inclinaison
- la mesure de vitesse et de distance de déplacement, comme pour des équipements sportifs (ex. montres)
- l'aide à la stabilisation de l'image, l'anti-flou, pour les appareils photo et caméras, etc…

• L'accéléromètre de la carte STM32F4-discovery (LIS3DSH) utilise **l'interface SPI** via les ports GPIO.
Le document joint *lis3dsh_datasheet.pdf* fournit les caractéristiques de ce périphérique. On y trouve notamment un descriptif pour **lecture et écriture** via cette interface, pages 26 et 27.

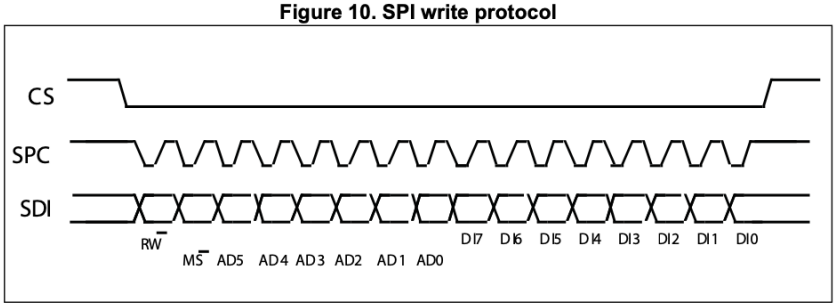
Une **lecture** est réalisée, après **sélection** de l'esclave via le slave select (CS ici), sur 16 cycles d'horloge. Le **bit correspondant à la lecture (valeur 1) suivi de l'adresse sur 7 bits** sont envoyés sur le MOSI (SDI ici), puis l'esclave transmet la **donnée sur 8 bits** sur le MISO (SDO ici), pendant que le maître transmet des 0 (on rappelle : transmission full duplex). Enfin l'esclave est **désélectionné**.

Figure 8. SPI read protocol



Une **écriture** est aussi réalisée, après **sélection** de l'esclave via le slave select, sur 16 cycles d'horloge. Le **bit correspondant à l'écriture (valeur 0) suivi de l'adresse sur 7 bits** sont envoyés sur le MOSI (SDI), suivis par la **donnée sur 8 bits**. Enfin l'esclave est **désélectionné**.

Figure 10. SPI write protocol



• Les signaux de cette interface SPI (interface **SPI1**) sont connectés aux ports GPIO comme suit :

- **SS** (c'est-à-dire CS) : port GPIOE, pin 3
- MOSI (c'est-à-dire SDI) : port GPIOA, pin 7
- MISO (c'est-à-dire SDO) : port GPIOA, pin 6
- SCLK (c'est-à-dire SPC) : port GPIOA, pin 5

• Les **adresses des registres** de ce composant sont données dans le document joint, dans le tableau situé aux pages 29-31. Nous en reproduisons un extrait ci-dessous. Vous consulterez le document si d'autres adresses de registres sont nécessaires.

Table 15. Register address map

Name	Type	Register address		Default	Comment
		Hex	Binary		
INFO1	r	0D	00001101	0010 0001	Information register 1
INFO2	r	0E	00001110	0000 0000	Information register 2
WHO_AM_I	r	OF	00001111	0011 1111	Who I am ID
CTRL_REG3	r/w	23	00100011	-	Control registers
CTRL_REG4	r/w	20	00100000	-	
CTRL_REG5	r/w	24	00100100	-	
CTRL_REG6	r/w	25	00100101	-	
STATUS	r	27	00100111	-	Status data register
OUT_T	r	0C	00001100	-	Temperature output
OFF_X	r/w	10	00010000	0000 0000	X-axis offset correction
OFF_Y	r/w	11	00010001	0000 0000	Y-axis offset correction
OFF_Z	r/w	12	00010010	0000 0000	Z-axis offset correction

On y voit par exemple que le registre WHO_AM_I (lecture seule) est à l'adresse 0x0F et a comme valeur 0x3F, que le registre CTRL_REG3 (lecture/écriture) est à l'adresse 0x23, ou que le registre STATUS est à l'adresse 0x27.

• Le **registre STATUS** permettra de savoir lorsque de nouvelles données sont disponibles. Son organisation est décrite page 35 du document :

Table 29. Status register

ZYXOR	ZOR	YOR	XOR	ZYXDA	ZDA	YDA	XDA

Nous utiliserons le **bit ZYXDA** (numéro 3, données disponibles sur les axes X, Y et Z).

Table 30. Status register description

ZYXOR	X, Y, and Z-axis data overrun. Default value: 0 (0: no overrun has occurred; 1: a new set of data has overwritten the previous set)
ZOR	Z-axis data overrun. Default value: 0 (0: no overrun has occurred; 1: new data for the Z-axis has overwritten the previous)
YOR	Y-axis data overrun. Default value: 0 (0: no overrun has occurred; 1: new data for the Y-axis has overwritten the previous)
XOR	X-axis data overrun. Default value: 0 (0: no overrun has occurred; 1: new data for the X-axis has overwritten the previous)
ZYXDA	X, Y, and Z-axis new data available. Default value: 0 (0: a new set of data is not yet available; 1: a new set of data is available)
ZDA	Z-axis new data available. Default value: 0 (0: new data for the Z-axis is not yet available; 1: new data for the Z-axis is available)
YDA	Y-axis new data available. Default value: 0 (0: new data for the Y-axis is not yet available; 1: new data for the Y-axis is available)
XDA	X-axis new data available. Default value: 0 (0: new data for the X-axis is not yet available; 1: new data for the X-axis is available)

Questions à traiter et remettre impérativement avant de commencer le TP.

Question 1.
Rappeler le **rôle** des signaux présents dans une interface SPI (i.e. que reçoivent-ils et à quoi servent-ils ?)

Question 2.
Quel est le **mode de transmission** via une interface SPI, et qu'implique ce mode de transmission ?

Question 3.
Dans ce TP, après configuration de l'accéléromètre, nous utiliserons un allumage des leds pour révéler visuellement les mesures prises sur l'accéléromètre.
Rappeler les **3 principales actions que notre programme itérera** indéfiniment dans le but d'obtenir ce comportement (cf cours slide 18).

Pour la suite récupérer le fichier spi.tar dans le moodle et placez-le dans le répertoire de votre choix. Désarchiver le contenu. Il contient une petite application pour ce TP, principalement située dans les fichiers *firmware.c* et *spi_mems.c*

Question 1.

La fonction *spi_mems_configure_pins()* permettant de *configurer les pins* des ports GPIO est donnée. Elle utilise notamment la fonction suivante de la bibliothèque *libopencm3* :

```
void gpio_mode_setup(uint32_t gpioport, enum gpio_mode mode,
                     enum gpio_pullup pullup,uint8_t gpios);
    → Configure a group of pins.
    Parameters:
    gpioport: GPIO register base address
    mode: pin mode
    pullup: pin pullup/pulldown configuration
    gpios: GPIO pin identifier.
```

Selon les informations données plus haut quant au port GPIO et numéro de pin utilisés pour le *signal Slave Select* reconnaître, dans l'appel à la fonction *gpio_mode_select* servant à leur configuration, les **macros** qu'il nous faudra utiliser ci-dessous pour référencer ce port et ce pin.

Dans cette première question, nous allons compléter les fonctions ***spi_lis_mems_select*** et

spi_lis_mems_deselect, qui correspondent respectivement à la sélection et à la désélection de l'esclave.

On note que la bibliothèque *libopencm3* offre les fonctions suivantes :

```
void gpio_clear(uint32_t gpioport,uint8_t gpios);
    → Clear one or more pins of the given GPIO port.
    Parameters:
    gpioport: GPIO register base address
    gpios: GPIO pin identifiers. Any combination of pins may be specified by
    OR'ing them together.
```

et

```
void gpio_set(uint32_t gpioport,uint8_t gpios);
    → Set one or more pins of the given GPIO port.
    Parameters:
    gpioport: GPIO register base address
    gpios: GPIO pin identifiers. Any combination of pins may be specified by
    OR'ing them together.
```

En utilisant ces fonctions, et les macros repérées ci-dessus, compléter les définitions des fonctions ***spi_lis_mems_select*** et ***spi_lis_mems_deselect***.

Question 2.

Nous allons maintenant compléter les deux fonctions essentielles de cette application, pour la lecture et l'écriture de données vers/dans des registres du périphérique, ce sont les fonctions ***spi_read_command*** et ***spi_write_command***.

La fonction ***spi_read_command*** prend en paramètre une adresse de registre dans lequel on veut lire, et renvoie la valeur lue. Elle procède comme décrit au début de l'énoncé, c'est-à-dire en substance :

- sélection de l'esclave (cf. question 1)
- écriture sur SPI1 d'un octet (penser à le former correctement !) : bit de lecture (valeur 1), adresse
  (et lecture sur la même interface d'une valeur - non pertinente - envoyée par l'esclave en full duplex)
- écriture sur SPI1 d'un octet de valeur non pertinente (typiquement 0) et lecture sur la même interface de la donnée - pertinente ici - envoyée par l'esclave
- désélection de l'esclave (cf. question 1).

La fonction ***spi_write_command*** prend en paramètre une adresse de registre dans lequel on veut écrire et la donnée à écrire. Elle procède comme décrit au début de l'énoncé, c'est-à-dire en substance :

- sélection de l'esclave
- écriture sur SPI1 d'un octet (penser à le former correctement !) : bit d'écriture (valeur 0), adresse
  (et lecture sur la même interface d'une valeur - non pertinente - envoyée par l'esclave en full duplex)
- écriture sur SPI1 de la donnée
  (et lecture sur la même interface d'une valeur - non pertinente - envoyée par l'esclave en full duplex)
- désélection de l'esclave.

On note que la bibliothèque *libopencm3* offre les fonctions suivantes :

```
void spi_send(uint32_t spi, uint16_t data);
  → écriture d'une donnée (qui sera 8 bits ici) sur une interface SPI
  (utiliser la macro SPI1 ici).
```

et

```
uint16_t spi_read(uint32_t spi);
  → lecture d'une donnée (qui sera 8 bits ici) sur une interface SPI
  (utiliser la macro SPI1 ici).
```

En utilisant ces fonctions, compléter les définitions des fonctions ***spi_read_command*** et ***spi_write_command***.

Question 3.
Avant d'aller plus loin, il est essentiel de s'assurer que ces deux fonctions sont bien opérationnelles. Pour **tester ces deux fonctions**, nous allons d'abord tester la *lecture* en lisant le contenu (connu) d'un registre, puis tester *l'écriture* en écrivant une valeur dans un registre et en relisant le contenu.
Pour ce faire, nous allons compléter le début de la fonction ***main*** dans le fichier firmware.c

Test de la fonction de lecture : nous avons noté plus haut que le registre WHO_AM_I (à l'adresse 0x0F) a comme valeur 0x3F.
Ajouter à la ligne 38 du fichier (point 1.) une *lecture* du contenu de ce registre WHO_AM_I, à placer dans la variable *v* prévue à cet effet.
Compiler et charger l'application. Placer un breakpoint à la suite de la ligne 38. Continuer l'exécution jusqu'à ce breakpoint et afficher dans gdb la valeur de *v*. Vérifier que vous avez bien lu la valeur 0x3F en hexadécimal (63 en décimal).
Si cela est correct, nous allons pouvoir passer au test de l'écriture.

Test de la fonction d'écriture : nous remarquons que le registre OFF_X (à l'adresse 0x10) est accessible en lecture/écriture. Nous allons l'utiliser pour ce test.
Ajouter à la ligne 40 du fichier (point 2.) une *écriture* de la valeur 0x22 dans ce registre, puis une *relecture* de la valeur de ce registre.
Compiler et charger l'application. Placer un breakpoint à la suite de la ligne 41. Continuer l'exécution jusqu'à ce breakpoint et afficher dans gdb la valeur lue. Vérifier que vous avez bien lu la valeur 0x22 en hexadécimal (34 en décimal).
Si cela est correct, nous pourrons remettre la valeur 0x00 dans ce registre, et continuer le développement/test de l'application.

Question 4.
La fonction *spi_write_command* étant opérationnelle, nous pouvons l'utiliser pour la fonction ***mems_init*** qui charge des valeurs d'initialisation (fournies), successivement dans les registres CTRL_REG5, CTRL_REG6, FIFO_CTRL, CTRL_REG4, et CTRL_REG3.

Compléter la définition de la fonction ***mems_init*** (en utilisant la documentation, ajouter les adresses de registres manquantes), décommenter cette fonction et son appel.

Question 5.
Après l'appel à cette fonction *mems_init*, la fonction *main* réalise une boucle infinie dans laquelle on attend qu'un nouvel échantillon soit disponible, on lit cet échantillon, et on met à jour l'allumage des leds en fonction de ces nouvelles données (fonction fournie).

Pour finaliser cette application, il nous reste à compléter les fonctions pour **l'attente** de la disponibilité d'un nouvel échantillon et pour la **lecture** des données.

5.1. Comme dit au début de l'énoncé, nous scruterons le bit ZYXDA du registre STATUS pour l'attente de la disponibilité de nouvelles données.

Compléter la définition de la fonction ***mems_wait_sample*** pour réaliser cette attente.

5.2. Enfin, la lecture des nouvelles données consistera à récupérer les contenus des **registres OUT_X_H et OUT_Y_H** (à placer dans les variables globales x et y).

Compléter la définition de la fonction ***mems_read_sample*** pour réaliser cette lecture (utiliser le tableau situé aux pages 29-31 déjà mentionné).

Il reste à vérifier le bon fonctionnement de l'application.
Compiler, charger, exécuter l'application.
Tester en tenant la carte horizontalement et en l'inclinant dans diverses directions, les leds doivent s'allumer pour indiquer l'inclinaison de la carte.
Faites contrôler ce bon fonctionnement par votre encadrant de TP.